Q9 *Deforestation* **4 points**

Original source: Fall 2022 Midterm 2 - Question 7 (originally an A+ bonus question) | **Level:** Challenge

Convert a non-empty `Tree` to a directed graph with one graph node per tree node and one edge from each parent to each child. Number nodes starting at 0 in preorder: visit the root, then each branch from left to right, applying the same rule recursively. The original tree labels do not affect these numbers.

The original question uses `Graph` from the source exam's Question 6. The needed definition is provided here. `add_edge` returns `None`.

```
class Graph:
    def __init__(self, nodes):
        self.nodes = nodes
        self.edges = []

    def add_edge(self, source, destination):
        if 0 <= source < self.nodes and 0 <= destination < self.nodes:
            if (source, destination) not in self.edges:
                self.edges.append((source, destination))
```

For `Tree('a', [Tree('b', [Tree('d')]), Tree('c')])`, the graph has four nodes and edges 0 -> 1, 1 -> 2, and 0 -> 3. Edge order does not matter. Complete the four expressions below without changing `helper`.

```
def tree_to_graph(tree):
    def helper(tree, op1, op2):
        a = op1()
        return op2(a, [helper(b, op1, op2) for b in tree.branches]) or a
    num_nodes = helper(tree, _______, _______)  # (a), (b)
    g = Graph(num_nodes)
    b = iter(range(num_nodes))
    helper(tree, _______, _______)  # (c), (d)
    return g
```

Q9.1 (2 points) Fill in blanks (a) and (b).

(a) `lambda: 1`

(b) `lambda a, b: a + sum(b)`

Q9.2 (1 point) Fill in blank (c).

(c) `lambda: next(b)`

Q9.3 (1 point) Fill in blank (d).

- ● `lambda a, b: any([g.add_edge(a, i) for i in b])`
- ○ `lambda a, b: all([g.add_edge(a, i) for i in b])`
- ○ `lambda a, b: [g.add_edge(a, i) for i in b]`
- ○ `lambda a, b: any([g.add_edge(i, a) for i in b])`
- ○ `lambda a, b: g.add_edge(a, b)`

Solution: The first traversal counts one for the current node plus the counts of its branches. The second assigns the next number before visiting any branches, which gives preorder numbering. Its `op2` adds edges to the child numbers. The list comprehension makes every `add_edge` call. Each returns `None`, so `any` returns `False` (also for a leaf), and `or a` returns the current node's number. Using `all` would return `True` for a leaf and corrupt the child numbers returned to its parent.